THE IRON NEXUS

Group # 6
Group Members (1): Raja M. Taha Ebaad 24i-0516

Module-by-Module Justification

Module 1: Train Registry (50 marks)

- **Correct Insertion (15 marks):** Uses an AVL Tree (insert() + balance()) for recursive BST insertion by TrainID. The balance() function checks the balance factor and triggers LL/RR/LR/RL rotations, guaranteeing O(log n) time even if IDs are inserted sequentially.
- **Correct Deletion (15 marks):** Uses remove() and replaces nodes with their in-order successor for two-child deletions. The tree rebalances during the recursion return, handling all leaf and child cases perfectly.
- **Search Operation (10 marks):** Employs binary search for O(log n) speed. An LRU-1 cache (cachedId/cachedNode) returns O(1) on repeat lookups, which is critical for repeated coach and seating access.
- **Display (10 marks):** Features Inorder (sorted by ID), Preorder (root-first for serialization), and Postorder (leaves-first) traversals. A visualizer prints the actual tree structure with branch characters.

Module 2: Coach Management (50 marks)

- **Attach at Front/End (12 marks total):** Uses a Doubly Linked List for O(1) insertions at both ends. Head and tail pointers are updated directly without traversing the list.
- **Insert/Delete (20 marks total):** Requires an O(n) walk to find the specific position or coach number, followed by O(1) unlinking or pointer updates. Boundary positions and all structural deletion cases are handled securely.
- **Traverse & Reverse (18 marks total):** Bidirectional traversal allows natural movement between the ENGINE and CABOOSE. Reversal operates in O(n) time and O(1) space by swapping the prev and next pointers on all nodes in-place.

Module 3: Route Management (50 marks)

- **Add/Remove Station (13 marks total):** An Adjacency Matrix (capped at 15 stations) allows O(1) station insertions. Station removal takes $O(V^2)$ to rebuild and compact the matrix while maintaining index integrity for remaining routes.
- **Add/Remove Route (13 marks total):** Operations are O(1) by updating the matrix indices directly for the undirected weighted graph.
- **Algorithms & Visualization (24 marks):** Implements Dijkstra ($O(V^2)$) to reconstruct exact shortest paths, BFS for level-order unreachability detection, and DFS for depth-first exploration. All algorithms produce colored, step-by-step visual output.

Module 4: Seat Management (50 marks)

- **Seat Booking & Search (Layer 1 - Hash Table):** Uses Knuth multiplicative hashing to distribute keys more uniformly than standard modulo, paired with linear probing. This guarantees the instant O(1) availability response required by the specs.
- **Cancellation (Layer 1):** Uses tombstone deletion (deleted=true) rather than clearing the slot, preserving probe chains for future lookups.
- **Insert & Delete (Layer 2 - BST):** Maintains a persistent ordered structure independent of the hash table. The cancellation function simply resets the booked flag to preserve the node for continuous ordered traversal.

Module 5: Logging System (50 marks)

- **Log Record (25 marks):** Implements a Linked-List Stack with O(1) LIFO pushes. Most recent operations appear first, complete with localtime() timestamps.
- **Proper Deletion (25 marks):** Allows an O(1) pop from the top for recent logs, or an O(n) indexed deletion while preserving accurate LIFO order and count.

Bonus Features & Engineering Constraints

- **Advanced Features:** Includes a dual-stack Undo/Redo system covering 11 action types, along with LRU-1 caching and Knuth hashing optimizations.
- **Data Serialization:** Preorder serialization guarantees exact AVL tree reconstruction. The Seat BST is saved in a secondary hierarchical format that visually shows parent-child relationships.
- **Memory & UI:** Zero STL containers are used. Destructors and a cleanupAll() function ensure all manual memory is freed correctly. The terminal interface utilizes ANSI escape codes and printf formatting for a structured, color-coded experience. System inputs are completely crash-proof against invalid data types.